

# PPT 구성안: 윗놀이 강화학습 프로젝트

이 문서는 발표 자료에 바로 옮길 수 있도록 프로젝트의 앞부분 구조를 슬라이드 단위로 정리한 것이다.

## Slide 1. 프로젝트 주제

제목: 윗놀이 강화학습 프로젝트

핵심 문장: 한국 전통 놀이인 윗놀이를 강화학습 환경으로 구현하고, 고정된 Rule-based Agent를 상대하는 PPO 기반 agent의 승률을 높이는 프로젝트이다.

포함 내용:

- 문제 유형: 2인 확률 기반 보드게임
- 목표: 고정 Rule-based Agent를 상대하는 PPO 모델 학습
- 최종 평가: 같은 규칙과 seed 조건에서 선공/후공을 바꿔 평가

## Slide 2. 프로젝트 목표

핵심 질문:

- 같은 윗놀이 규칙에서 PPO가 Rule-based Agent를 이길 수 있는가?
- state/action/reward 설계가 PPO 성능에 어떤 영향을 주는가?
- 윗 결과의 randomness와 선공 이점을 줄이고 같은 조건에서 승률을 비교할 수 있는가?

역할 정리:

| 역할         | 내용                                        |
|------------|-------------------------------------------|
| MyAgent    | 고정 Rule-based Agent, PPO가 이겨야 하는 baseline |
| TeamPPO    | MaskablePPO 기반 강화학습 모델                    |
| Common Env | 두 agent를 같은 규칙과 seed에서 평가하는 환경            |

## Slide 3. 환경 및 데이터셋

핵심 문장: 정적 데이터셋을 사용하지 않고, 윗놀이 시뮬레이션 환경에서 episode를 생성해 학습한다.

포함 내용:

- Dataset 없음
- self-play / opponent-play 기반 simulated episode 사용
- 한 episode = 윗놀이 한 판
- 말 4개가 모두 FINISHED 상태가 되면 episode 종료

state -> action -> environment transition -> reward -> next state

## Slide 4. 공통 게임 규칙

### 환경 구성:

- 플레이어 수: 2명
- 플레이어당 말: 4개
- 잡기와 업기 사용
- 옷/모 추가 턴 사용
- 잡기 후 추가 턴 사용
- 지름길 자동 진입
- HOME 통과 시 FINISHED

### 제외 규칙:

- 뒷도 없음
- 낙 없음
- 후진 이동 없음
- 플레이어의 지름길 선택 없음
- 옷/모 20회 연속 즉시 승리 없음

## Slide 5. 옷 확률

| 결과 | 이동량 | 확률     |
|----|-----|--------|
| 도  | 1   | 0.1536 |
| 개  | 2   | 0.3456 |
| 걸  | 3   | 0.3456 |
| 옷  | 4   | 0.1296 |
| 모  | 5   | 0.0256 |

**발표 포인트:** 모든 학습 및 평가에서 같은 옷 확률을 사용해 확률 환경을 통제했다.

## Slide 6. Seed와 Episode 생성

### Paired evaluation 방식:

base seed 1개당 2게임

Game A: PPO 선공, Rule-based Agent 후공

Game B: Rule-based Agent 선공, PPO 후공

### 평가 규모:

- paired seed 수: 2,500
- PPO seed별 평가 게임 수: 5,000
- 3개 학습 seed 기준 총 15,000판

**발표 포인트:** 같은 seed에서 선공과 후공을 모두 수행해 선공 편향을 줄였다.

## Slide 7. Preprocessing

환경의 raw game state를 PPO 입력 vector로 변환한다.

| 단계               | 설명                                             |
|------------------|------------------------------------------------|
| 위치 인코딩           | 말 위치와 상태를 vector로 변환                           |
| stack 인코딩        | 업힌 말 정보를 matrix로 표현                            |
| pool 인코딩         | 현재 사용할 수 있는 윗 결과 개수                            |
| action mask      | 가능한 action만 선택하도록 mask 생성                      |
| tactical feature | action별 capture, finish, distance, rf_score 계산 |

## Slide 8. Action 설계

```

action = piece_id * 5 + yut_id

piece_id: 0~3
yut_id: 0~4
총 action 수: 20

```

예시:

- `piece_id=2, yut_id=3`이면 2번 말을 윗 결과로 이동
- 현재 pool에 해당 yut 결과가 없으면 illegal action

**발표 포인트:** 매 state마다 legal action이 달라지므로 action masking이 중요하다.

## Slide 9. Legal Action Mask

역할:

- 불가능한 행동을 policy 선택에서 제외
- PPO logits에서 illegal action을 매우 작은 값으로 masking
- illegal action으로 인한 학습 불안정 감소

**Illegal action 예시:**

- pool에 없는 윗 결과 사용
- 이미 FINISHED인 말 이동
- stack 대표 말이 아닌 말을 선택

## Slide 10. Base State

base state는 게임에서 직접 관측 가능한 정보로 구성된다.

| 정보          | 설명              |
|-------------|-----------------|
| 내 말 위치      | 4개 말의 위치와 상태    |
| 상대 말 위치     | 상대 4개 말의 위치와 상태 |
| stack 정보    | 같은 칸에 업힌 말 정보   |
| pool count  | 현재 보유한 윗 결과     |
| action mask | 현재 가능한 action   |

**한계:** capture, finish, 위험도 같은 전술 정보를 모델이 직접 추론해야 한다.

## Slide 11. Tactical State

tactical state는 base state에 action-level 전략 feature를 추가한다.

| Feature        | 의미                             |
|----------------|--------------------------------|
| capture        | 상대 말을 잡는가                      |
| captured_count | 몇 개의 말을 잡는가                    |
| finish         | 완주가 발생하는가                      |
| finished_count | 몇 개의 말이 완주하는가                  |
| stack_size     | 함께 움직이는 말 수                    |
| distance_after | 이동 후 남은 거리                     |
| rf_score       | Rule-based Agent의 action 평가 점수 |

**발표 포인트:** 최종 모델은 원본 상태만 사용하는 Pure RL이라기보다 RL + State Engineering으로 보는 것이 정확하다.

## Slide 12. Reward 설계

### Terminal Reward

```

승리: +1
패배: -1
그 외 step: 0

```

**장점:** 최종 목표인 승패와 직접 일치

**단점:** reward가 늦게 들어와 credit assignment가 어려움

### RF-shaped Reward

```

capture bonus: +0.08 * captured_count
finish bonus: +0.15 * finished_count
shortcut bonus: +0.02

```

opponent capture penalty  
opponent finish penalty

**실험 해석:** reward shaping은 capture 성향을 강화했지만, 최종 승률을 항상 개선하지는 않았다.

## Slide 13. 서로의 설계 연결

**핵심 문장:** 내가 만든 Rule-based Agent는 단순 평가 상대가 아니라, PPO가 학습해야 할 전략 기준과 tactical feature 설계의 출발점이 되었다.

| 구성          | 내 프로젝트 / MyAgent                                      | 팀원 PPO 프로젝트                                                |
|-------------|-------------------------------------------------------|------------------------------------------------------------|
| Agent 역할    | 고정 Rule-based opponent                                | PPO로 opponent를 이기는 agent                                   |
| 전략 기준       | 잡기, 완주, 업기, 남은 거리 기반 점수화                              | 해당 기준을 tactical observation feature로 일부 반영                 |
| State 설계 영향 | can capture, can finish, danger, distance 같은 판단 기준 제공 | capture, finish, distance_after, rf_score를 observation에 추가 |
| 평가 역할       | PPO가 이겨야 할 baseline                                   | baseline 상대 승률을 높이는 학습 대상                                  |

**발표 포인트:** TeamPPO가 MyAgent를 imitation learning으로 그대로 따라 배운 것은 아니다. 하지만 MyAgent의 판단 기준이 state engineering과 evaluation baseline에 반영되었다.

## Slide 14. State / Reward 설계 비교 실험

3M candidate sweep에서 observation과 reward 조합을 비교했다.

| Observation | Reward    | 평균 승률        |
|-------------|-----------|--------------|
| base        | terminal  | 37.4%        |
| base        | rf_shaped | 32.4%        |
| tactical    | terminal  | <b>53.3%</b> |
| tactical    | rf_shaped | 52.3%        |

**해석:**

- base state만 사용하면 capture/finish/distance를 PPO가 직접 추론해야 해서 성능이 낮았다.
- tactical state를 추가하자 평균 승률이 **37.4% -> 53.3%**로 크게 상승했다.
- rf\_shaped reward는 capture 성향은 강화했지만 전체 승률은 오히려 낮아졌다.
- 최종 장기 학습 후보는 **tactical + terminal**로 선택했다.

## Slide 15. Hyperparameter / 학습 설정 튜닝

**튜닝 방향:**

- observation mode: **base -> tactical**
- reward mode: **rf\_shaped** 비교 후 **terminal** 선택

- action handling: MaskablePPO + legal action mask
- training budget: 3M -> 10M -> 30M -> 40M
- parallel env: 장기 학습에서 **SubprocVecEnv**, **n\_envs=12** 사용
- seed: 0, 1, 2의 3개 학습 seed 사용

| 단계                | 설정                                    | 평균 승률  | 의미          |
|-------------------|---------------------------------------|--------|-------------|
| 3M sweep          | tactical + terminal                   | 53.3%  | 후보 선정       |
| 10M long training | tactical + terminal                   | 57.95% | 학습량 증가 효과   |
| 30M common paired | tactical + terminal                   | 56.49% | 공통 평가 기준 적용 |
| 40M retraining    | tactical + terminal + common opponent | 59.76% | 목표 60%에 근접  |

**발표 포인트:** 성능 향상은 단일 알고리즘 변경보다 state 설계, 평가 기준 정렬, 학습량 증가가 함께 작용한 결과다.

## Slide 16. 설계 변경으로 얻은 결론

| 질문                      | 결론                                                             |
|-------------------------|----------------------------------------------------------------|
| State 설계가 중요한가?         | 매우 중요했다. tactical feature 추가가 가장 큰 성능 향상을 만들었다.                |
| Reward shaping이 항상 좋은가? | 아니었다. rf_shaped는 capture 성향을 강화했지만 최종 승률은 낮아졌다.                |
| Action mask가 필요한가?      | 필요하다. 윗놀이에서는 매 state마다 가능한 action이 달라진다.                       |
| Pure RL인가?              | 원본 상태만 사용하는 Pure RL은 아니며, RL + State Engineering으로 보는 것이 정확하다. |
| MyAgent의 역할은?           | PPO가 이겨야 할 baseline이자 tactical feature 설계의 기준 역할을 했다.          |

**핵심 결론:** 윗놀이 강화학습에서는 알고리즘 자체보다도 state/action/reward 설계와 공정한 평가 프로토콜이 성능 신뢰도를 크게 좌우했다.

## Slide 17. 최종 설계 요약

| 구성 요소           | 최종 선택                    |
|-----------------|--------------------------|
| Algorithm       | MaskablePPO              |
| State           | tactical observation     |
| Action          | 20 discrete actions      |
| Action handling | legal action mask        |
| Reward          | terminal reward          |
| Evaluation      | common paired evaluation |

**핵심 결론:** 윗놀이 RL에서는 알고리즘 자체뿐 아니라 state/action/reward 설계가 성능에 큰 영향을 주었다.

## Slide 18. 알고리즘 및 주요 Hyperparameter

| 항목              | 설정                |
|-----------------|-------------------|
| Algorithm       | MaskablePPO       |
| Policy          | MlpPolicy         |
| Observation     | tactical          |
| Reward          | terminal          |
| Opponent        | common_rule_based |
| Training seeds  | 0, 1, 2           |
| Total timesteps | seed별 40M         |
| Vector env      | SubprocVecEnv     |
| Number of envs  | 12                |
| Device          | CUDA              |
| Learning rate   | 3e-4              |
| n_steps         | 2048              |
| Batch size      | 64                |
| Gamma           | 0.99              |
| GAE lambda      | 0.95              |

**발표 포인트:** 성능 개선은 learning rate 하나를 바꾼 결과가 아니라, observation/reward/training budget/evaluation protocol을 단계적으로 조정한 결과다.

## Slide 19. Evaluation Metric

| 지표               | 의미                     |
|------------------|------------------------|
| 전체 승률            | 전체 5,000판 중 PPO가 이긴 비율 |
| 선공 승률            | PPO가 선공일 때의 승률         |
| 후공 승률            | PPO가 후공일 때의 승률         |
| seed 평균          | 여러 학습 seed의 평균 성능      |
| seed 표준편차        | 학습 안정성                 |
| illegal action   | 불가능한 행동 선택 여부          |
| evaluation error | 비정상 종료 여부              |

**발표 포인트:** 단순 전체 승률뿐 아니라 선공/후공, seed 편차, illegal action을 함께 확인해 결과 신뢰도를 높였다.

## Slide 20. 최종 결과 1: Common Rule-Based 기준

| TeamPPO seed | 전체 승률  | 선공     | 후공     | 통과 여부 |
|--------------|--------|--------|--------|-------|
| 0            | 58.42% | 59.00% | 57.84% | false |
| 1            | 60.46% | 60.76% | 60.16% | true  |
| 2            | 60.40% | 61.32% | 59.48% | true  |

3-seed 평균 승률: 59.76%  
 60% 통과: 2 / 3 seeds  
 illegal actions: 0  
 evaluation errors: 0

해석: 공통 rule-based 기준에서는 목표 60%에 거의 도달했다.

## Slide 21. 최종 결과 2: MyAgent 직접 평가

| TeamPPO seed | TeamPPO 승률 | MyAgent 승률 |
|--------------|------------|------------|
| 0            | 57.26%     | 42.74%     |
| 1            | 57.56%     | 42.44%     |
| 2            | 55.08%     | 44.92%     |

TeamPPO 평균 승률: 56.63%  
 MyAgent 평균 승률: 43.37%  
 총 게임 수: 15,000  
 illegal actions: 0  
 evaluation errors: 0

해석: MyAgent 직접 평가에서는 TeamPPO가 우세했지만, 60% 목표에는 도달하지 못했다.

## Slide 22. 한계 및 개선 방향

한계:

- 최종 모델은 원본 상태만 사용하는 Pure RL이 아니라 RL + State Engineering이다.
- common\_rule\_based 기준과 MyAgent 직접 평가 기준의 승률이 다르다.
- reward shaping은 기대와 달리 최종 승률 개선으로 이어지지 않았다.
- 5,000판 평가도 확률 게임 특성상 seed 영향이 완전히 사라지지는 않는다.

개선 방향:

- 더 많은 학습 seed와 평가 seed 사용
- confidence interval을 그래프와 함께 제시
- 32M, 36M, 40M checkpoint 비교
- tactical feature 중 어떤 feature가 중요한지 ablation
- MyAgent 기준으로 common env에서 재학습



## Slide 23. 최종 결론

### 결론:

- 윗놀이 강화학습에서는 state/action/reward 설계가 성능에 큰 영향을 준다.
- tactical observation은 base observation보다 훨씬 좋은 성능을 보였다.
- terminal reward가 rf\_shaped reward보다 최종 승률 측면에서 안정적이었다.
- TeamPPO는 common rule-based 기준 평균 59.76%, MyAgent 직접 기준 평균 56.63%를 기록했다.
- MyAgent는 PPO가 이겨야 할 기준선이자 tactical feature 설계의 기준 역할을 했다.